Trippy: An AI-Powered Travel Planning Web Application

Abstract

Trippy is an AI-assisted travel planning web application built to simplify the process of creating and refining personalized itineraries. The system combines a React-based frontend, an Express backend API, and a Gemini-powered AI orchestration layer (via LangChain). Users provide trip constraints such as origin, destination(s), dates, and budget, then receive itinerary guidance and a day-by-day plan. A major feature is conversational refinement through chat with streaming responses, allowing users to iteratively modify plans in near real time. The project demonstrates how large language models can be integrated into interactive software while addressing practical concerns such as prompt design, response parsing, validation, and user trust.

1. Introduction and Problem Statement

Planning a trip often requires combining logistics, budgeting, local recommendations, and scheduling across multiple tools. Traditional travel platforms can provide booking options and static recommendations, but they usually do not offer dynamic, conversational customization end to end. Large language models (LLMs) offer a new approach: users can describe goals and constraints in natural language, while the system generates and revises plans interactively. However, turning this into a usable product requires more than model access. It requires:

- Input validation and constraint handling
- Structured output for UI rendering
- Real-time interaction patterns
- State consistency between chat and itinerary views

Trippy addresses this by providing a unified workflow where users can enter structured trip details, get an initial generated plan, and then refine that plan through an AI chat interface.

2. Objectives

The project objectives are:

1. Reduce travel-planning effort through AI-assisted automation.
2. Translate user constraints into practical, structured itinerary suggestions.
3. Support iterative refinement through conversational interaction.
4. Improve perceived responsiveness with streaming model output.
5. Provide a clear and usable interface for planner input, saved trips, and itinerary review.

3. System Architecture and Design

Trippy uses a monorepo organization with three primary parts:

3.1 Backend (backend/)

Node.js + Express API layer responsible for:

- Health checks
- Chat request validation
- Non-streaming and streaming AI endpoints
- Bridging frontend message format to LangChain message format

3.2 Frontend (web/)

React + Vite client responsible for:

- Trip planner and validation
- Google Places autocomplete UX
- Saved trips and local persistence
- Chat and itinerary rendering

3.3 AI Layer (gemini-agent/)

Shared AI core responsible for:

- System prompt/persona behavior

- Gemini model invocation through LangChain
- Retry/backoff and optional fallback model logic
- Shared behavior across API and CLI usage

3.4 Data Flow Overview

1. User enters planner details in the frontend.
2. Frontend stores trip state and navigates to results page.
3. Frontend sends chat/history messages to backend.
4. Backend validates and forwards transformed messages to Gemini (LangChain).
5. Response is returned either fully (/api/chat) or as stream chunks (/api/chat/stream).
6. Frontend updates chat incrementally and attempts to extract itinerary JSON for structured display.

This design separates responsibilities and makes the system easier to evolve.

4. Implementation Details

4.1 Frontend Implementation

The frontend is built with React 19 and Vite. Key implementation elements include:

- Planner route (/) with:

        - Origin (travellingFrom)
        - Primary and additional destinations (up to 10)
        - Destination kind toggle (city or landmark)
        - Start/end date selection
        - Flexible date option
        - Budget amount and budget mode (per person or total)
    - Results route (/trip/:tripId) with split-pane UI:
        - Left: ChatPanel
        - Right: ItineraryPanel
    - Theme toggle (light/dark)
    - Google Places autocomplete integration
    - Local persistence using localStorage

State is managed using a custom context + reducer (TripsProvider) with normalized shape (tripsById, tripIds) and patch-style updates.

4.2 Backend Implementation

The backend uses Express 5 with CORS and JSON parsing middleware. Key endpoints:

- GET /api/health
    - Returns API availability and whether Gemini is configured

- POST /api/chat
    - Returns full assistant reply in one response
- POST /api/chat/stream
    - Streams text chunks over Server-Sent Events (SSE)

The backend validates message arrays, role values, and request shape before invoking the model. It also transforms client messages into LangChain message objects.

4.3 AI Integration

The AI layer uses:

- @langchain/google (ChatGoogle)
- @langchain/core/messages
- Gemini model configuration via environment variables

A detailed system prompt defines Trippy's behavior and response style. For itinerary generation, the app prompts the model to provide:

1. Human-readable guidance
2. A JSON itinerary block with days and items

Important: the app requests strict JSON formatting and then parses it when present; it does not fully guarantee model compliance on every response.

5. Key Algorithms and Logic Flow

5.1 Streaming Response Handling

Streaming is implemented through SSE:

- Backend emits data: { text: "..." } events
- Frontend accumulates chunks and refreshes assistant message content incrementally
- End-of-stream is marked with [DONE]

This improves responsiveness and user feedback compared with waiting for full completion.

5.2 Itinerary JSON Extraction

The frontend attempts to parse fenced JSON blocks from assistant output (tryExtractItineraryJson). If valid JSON with expected structure (days array) is found, it updates structured itinerary state used by ItineraryPanel. This approach allows mixed natural language + machine-readable data in one model response.

5.3 Resilience Logic

The shared AI core includes retry/backoff logic for retryable failures (for example overload/rate-limit scenarios) and optional fallback model behavior if configured. This improves robustness under transient provider issues.

6. Testing Strategy

6.1 Current Testing

Current validation is primarily practical/manual:

- Planner and results flow verification
- Endpoint behavior checks (/api/health, chat endpoints)
- Streaming behavior verification in UI
- Itinerary rendering checks from model output

6.2 Recommended Improvements

To strengthen reliability, add:

- Unit tests for reducer/state transitions
- Unit tests for message transformation and JSON extraction
- API integration tests for endpoint validation and error states
- End-to-end tests for planner → generation → chat refinement flow
- Contract tests for expected itinerary JSON shape

7. Challenges and Solutions

7.1 AI Output Variability

- **Challenge:** LLM responses may vary in format and quality.
- **Solution:** Strong prompt structure + parser logic + fallback display behavior.

7.2 Streaming UI Consistency

- **Challenge:** Partial updates can cause flicker or stale state.
- **Solution:** Incremental patch updates, streaming placeholders, and controlled refresh timing.

7.3 Chat/Itinerary Synchronization

- **Challenge:** Keeping narrative chat and structured itinerary aligned.
- **Solution:** Shared trip store and update patches that can include both message and itinerary updates.

7.4 External Service Reliability

- **Challenge:** AI provider or network instability.
- **Solution:** Retry/backoff strategy and clear health/config messaging in UI.

8. Limitations

Current limitations include:

- Browser-local persistence only (localStorage)
- No user authentication or multi-device synchronization
- Limited automated test coverage
- Dependence on model output quality and format adherence
- No full production hardening yet (for example, advanced monitoring/rate limiting/deployment controls)

9. Future Improvements

Potential next steps:

1. Add backend persistence (PostgreSQL/MongoDB)
2. Add authentication and per-user trip storage
3. Add stronger structured-output validation/repair layer
4. Introduce caching and performance telemetry
5. Add richer itinerary editing (drag/drop, reorder, pin favorites)
6. Support collaboration/sharing workflows
7. Harden deployment and operational controls

8. Expand recommendation quality with feedback loops

10. Ethical Considerations

AI-based travel assistance creates user-trust responsibilities:

- **Reliability:** AI may produce outdated or incorrect details.
- **Misinformation risk:** Users may rely on generated recommendations.
- **Transparency:** Users should know content is AI-generated and may require verification.

Mitigations include clear disclaimers, encouraging user verification of critical details, and progressively improving validation.

11. Security Considerations

Key security points:

- Keep API keys in environment files, never in source control
- Validate request schema and message roles server-side
- Restrict backend exposure and apply operational protections in deployment
- Add rate limiting and abuse controls as production hardening steps

12. How to Run the Project

Prerequisites

- Node.js installed
- Valid Gemini API key configured in environment variables
- Google Places API key for autocomplete features

Commands

From repository root:

- npm run dev — run web + API together
- npm run api — run backend only
- npm run web — run frontend only
- npm run agent — run terminal-based Trippy CLI

Ensure environment variables are configured before startup.

13. Conclusion

Trippy demonstrates a practical pattern for integrating LLMs into interactive web software: structured planner input, streamed conversational refinement, and machine-readable itinerary rendering. The project's modular frontend/backend/AI split supports maintainability and future extension. While it is not yet production-complete, it provides

a strong functional foundation and highlights the key trade-offs in AI-driven application design, including output variability, validation strategy, and trust-oriented UX decisions